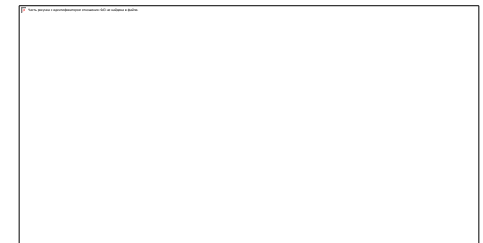
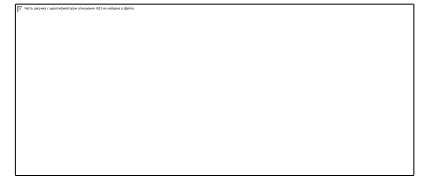


[Верстка лендинга]

[Организация рабочего процесса]



[Название курса]

Автор курса

Front-end разработчик в
компании Sperotek. Наставник он-лайн школ
с 2015 года. Автор видеоуроков



[Рубец Сергей]

[Название курса]

Тема

[Организация рабочего процесса]

Используемые технологии

1. Репозиторий на github
2. Препроцессор pug
3. Препроцессор sass
4. Таск менеджер gulp

Git – система контроля версий

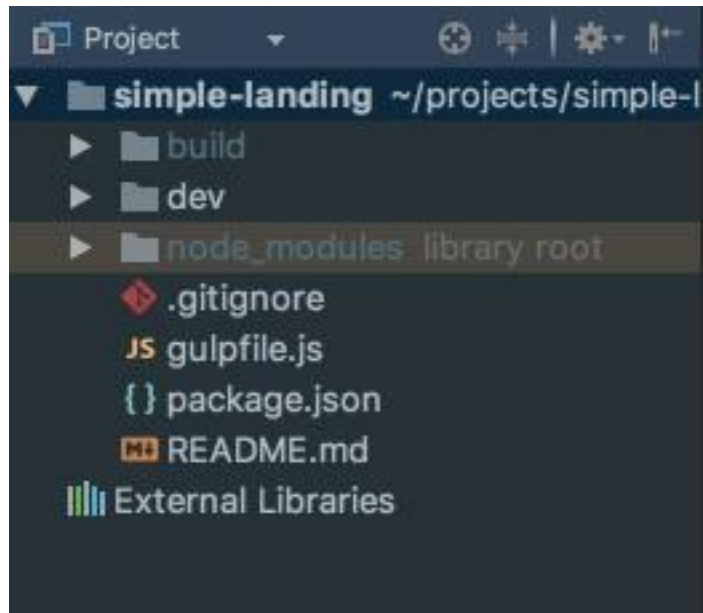
Система контроля версий — это система, регистрирующая изменения в одном или нескольких файлах с тем, чтобы в дальнейшем была возможность вернуться к определённым старым версиям этих файлов.

Например, вы работаете с кодом, сделали уже не малое кол-во функционала. Но в какой-то момент, один из них сломался. Пусть это будет слайдер, все было хорошо, но в один день он перестал работать. С помощью системы контроля версий мы можем посмотреть что было вчера или позавчера, найти тот момент, когда слайдер работал. После чего произвести анализ последующей работы, которая привела к поломке.

Это лишь один из примеров, как гит помогает работе. Почти всегда на сегодняшний день разработчики используют его для работы в командах, так как с помощью него, работа получается очень эффективная.

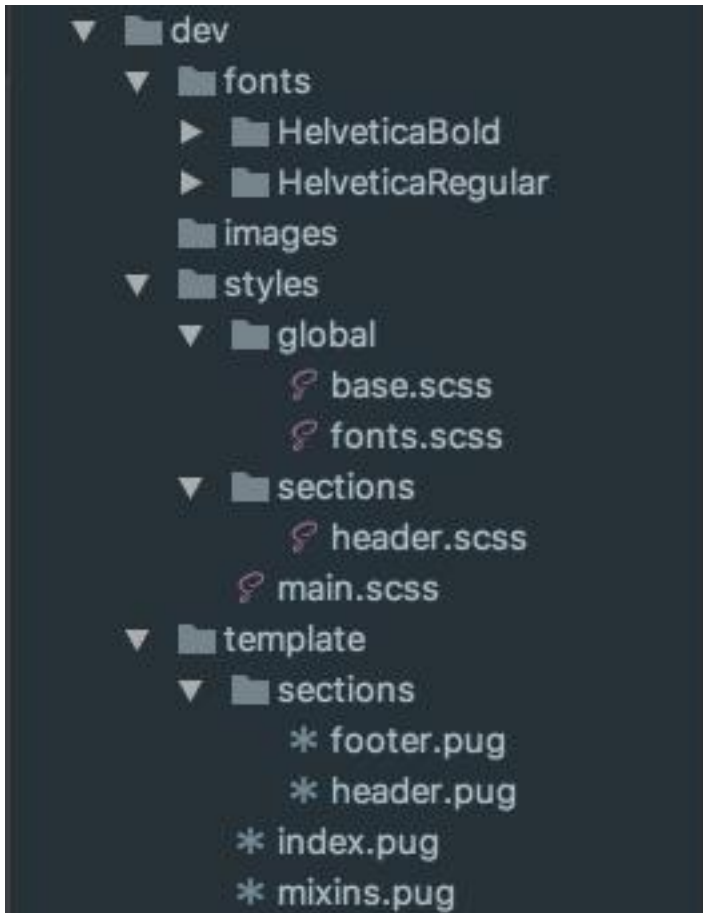
Один разработчик сделал слайдер, второй – параллакс, после чего отправили свои изменения на гит и вот вся команда может работать уже со слайдером и с параллаксом.

Структура проекта



1. Папка dev – в ней находятся все исходные файлы для разработки, а именно – pug шаблоны, scss стили, js код, картинки, шрифты, прочее. Эта папка важна разработчикам, она же заливается на git.
2. Папка build – здесь находятся все файлы, которые являются результатом компиляции, конкатинации, минификации и обработки с папки dev. В ней разработка не ведется, она нужна только для того, что б выложить ее на сервер

Dev папка

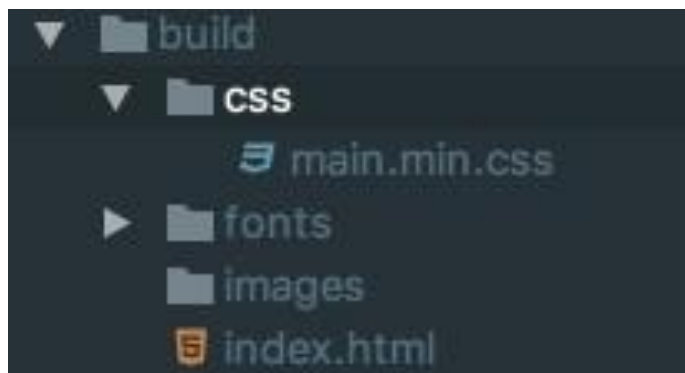


Template – наша разметка, которая разбивается по секциям. Очень удобно переиспользовать один и тот же код, например, у нас есть куча страничек, у каждой из них почти наверняка одинаковый header, footer. Что б не копипастить код, создаем отдельные соответствующие файлы и с помощью include подключаем их.

Styles – аналогично с разметкой. Для каждой секции будем создавать свой файл стилей, для header отдельный файл, для footer – отдельный и так для всех других секций

Так же в папке dev будут все необходимы дополнительные файлы: шрифты, картинки, иконки, js

Build папка



На выходе получаем один файл с разметкой, один сжатый файл со стилями, один сжатый файл js.

Так же, если у нас будут дополнительные файлы стилей и js сторонних библиотек, мы их будем выводить в отдельный файл, например, vendor.min.css и vendor.min.js

На практике обычно разделяют свои стили (скрипты) и сторонних библиотек, по соображению ускорении работы (компиляции файлов) и загрузки странички в браузере

Package.json

Файл "манифест", предназначенный для описания зависимостей проекта, а так же некоторые ведомости о нем.

Это особо полезно, так как другие разработчики, начавшие работать с нашим проектом не знают какие библиотеки используются в нашем проекте. Им достаточно набрать в консоле команду "npm install" и все библиотеки указанные в этом файле установятся в проект.

Так же здесь можно увидеть версию, имя проекта, описание, ключевые слова для поиска, имя автора и т.д.

```
scripts start
1 {
2   "name": "simple-landing",
3   "version": "1.0.0",
4   "description": "",
5   "main": "gulpfile.js",
6   "scripts": {
7     "start": "gulp"
8   },
9   "author": "",
10  "license": "ISC",
11  "devDependencies": {
12    "autoprefixer": "^6.7.5",
13    "browser-sync": "^2.18.8",
14    "gulp": "github:gulpjs/gulp#4.0",
15    "gulp-postcss": "^6.3.0",
16    "gulp-pug": "^3.2.0",
17    "gulp-rename": "^1.2.2",
18    "gulp-sass": "^3.1.0",
```

Gulp.js

Gulp – это таск-менеджер, предназначен для автоматизации разных рутинных задач, например, таких как: минификации, конкатинации файлов, добавление вендорных префиксов в css стилях, сжатия картинок, создание спрайтов, компиляции из препроцессорных языков и т.д.

Тут все зависит от ваших задач и желаний. По сути, сам по себе gulp не умеет делать выше указанные действия. Для этого необходимо устанавливать дополнительные плагины к нему, к счастью их существует огромное множество.

Например, мы хотим из sass компилировать в css, для этого достаточно подключить gulp-sass плагин. Либо же мы хотим сжать все картинки, находящиеся в папке images, для того, что б они не занимали слишком много места на диске, для этого есть плагин gulp-imagemin.

Все все плагины можно найти на этом сайте <https://www.npmjs.com> правда помимо плагинов для gulp, здесь находятся плагины для grunt(аналог gulp), а так же, различные node приложения, да и просто полезные файлы (тот же normalize.scss)

Gulpfile.js

Для того что б настроить те или иные задачи существует специальный файл конфигурации, он называется gulpfile.js

Все что что мы хотим сделать мы будем описывать как раз таки в этом файле, после чего запуская gulp, он нам выполнит наши желания.

Идея такая:

Создаем необходимую задачу, пишем ее имя и собственно функцию, которая будет вызываться.

В этой функции указываем путь к исходным файлам, далее делаем какие-то преобразования, после чего указываем куда положить файлы, которые получаются на выходе.

```
22
23  /* ----- Pug compile ----- */
24  gulp.task('templates-compile', function template() {
25    return gulp.src('dev/template/index.pug')
26      .pipe(pug({
27        pretty: true
28      }))
29      .pipe(gulp.dest('build'));
30  });
31
32  /* ----- Styles compile ----- */
33  gulp.task('styles-compile', function() {
34    return gulp.src('dev/styles/main.scss')
35      .pipe(sourcemaps.init())
36      .pipe(sass({outputStyle: 'compressed'}).on('error', sass.logError))
37      .pipe(postcss([autoprefixer()]))
38      .pipe(rename('main.min.css'))
39      .pipe(sourcemaps.write('./maps'))
40      .pipe(gulp.dest('./build/css'));
41  });
```